

**Web Applications
Workshop 5**

**Semester II 2026
Oct 1, 2026
Prof. Francisco Hidrobo**

Building and Testing a REST API with FastAPI, JWT, OpenAPI, and Social Authentication

Technology: Python + FastAPI

API format: REST/JSON

Documentation: OpenAPI / Swagger UI

Authentication: JWT + OAuth 2.0 social login

Testing tools: Swagger UI and Postman

Recommended database: MySQL/MariaDB or PostgreSQL

1. Workshop objective

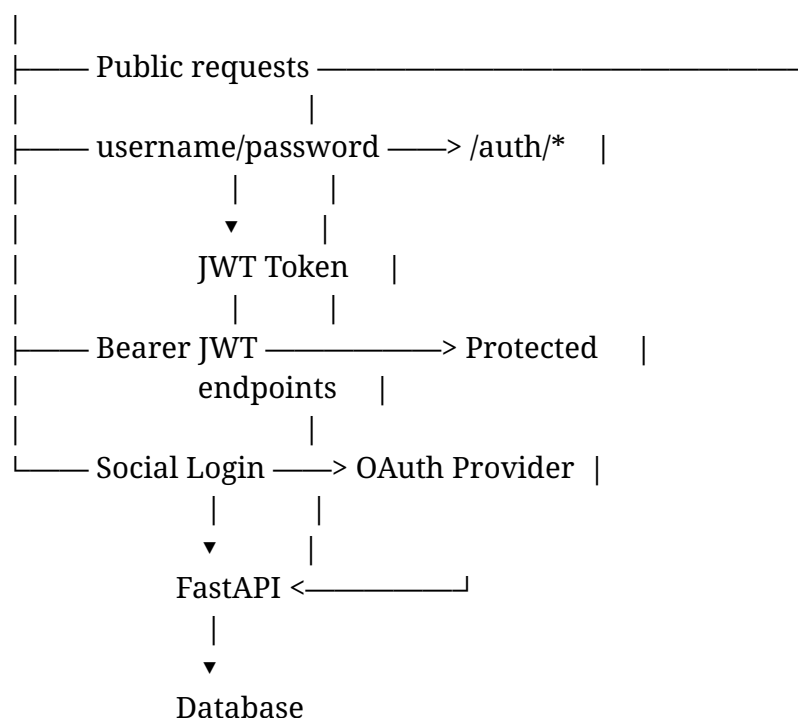
In this workshop, you will design, implement, document, secure, and test a REST API using FastAPI.

Your API must provide public and protected endpoints, user registration and authentication using JSON Web Tokens (JWT), token refresh, logout, a **Remember Me** mechanism, and registration/login using at least one external social identity provider.

You will also design one or more application-specific entities and implement CRUD operations protected by JWT authorization.

At the end of the workshop, your API should approximately have this structure:

Client



2. Functional requirements

Your group must implement an API containing:

1. **Public endpoints** that can be accessed without authentication.
2. **User registration** using username/email and password.
3. **Login** using username/email and password.
4. **JWT access tokens**.
5. **Refresh tokens**.
6. **Logout**.
7. **Remember Me** functionality.
8. A `/users/me` endpoint.
9. At least one **social authentication provider**, such as Google or GitHub.
10. One or more application-specific entities.
11. At least one complete CRUD implementation.
12. JWT-protected endpoints.
13. JSON request and response bodies.
14. Automatic OpenAPI documentation.
15. Testing using Swagger UI.
16. Testing using Postman.

Part I — Prepare the project

3. Create your Python environment

Create a directory for your project:

```
mkdir fastapi-workshop
cd fastapi-workshop
```

Create a virtual environment:

```
python -m venv venv
Linux/macOS:
```

```
source venv/bin/activate
Windows PowerShell:
```

```
.\venv\Scripts\Activate.ps1
```

Install the main dependencies:

```
pip install fastapi uvicorn sqlalchemy pymysql \
python-jose[cryptography] passlib[bcrypt] \
python-multipart pydantic[email] authlib httpx \
python-dotenv
```

Create a `requirements.txt`:

```
pip freeze > requirements.txt
```

Part II — Organize your API

4. Create the project structure

Use a modular structure rather than putting the entire API in one file.

```
fastapi-workshop/
|
|—— app/
|   |—— __init__.py
|   |—— main.py
|   |—— database.py
|   |—— models/
|       |—— __init__.py
|       |—— user.py
|       |—— item.py
|   |—— schemas/
|       |—— __init__.py
|       |—— user.py
|       |—— item.py
|   |—— routers/
|       |—— __init__.py
|       |—— auth.py
|       |—— public.py
|       |—— items.py
|   |—— security/
|       |—— __init__.py
|       |—— jwt.py
|
|—— .env
|—— requirements.txt
|—— README.md
```

Your group can replace Item with an entity appropriate to your project, for example:

Book
 Product
 Course
 Reservation
 Equipment
 Laboratory
 Event
 Patient
 Appointment
 Task
 Order
 ResearchProject

You may create multiple related entities if appropriate.

Part III — Configure FastAPI

5. Create the application

Create app/main.py:

```
from fastapi import FastAPI
```

```
app = FastAPI(
    title="FastAPI Workshop API",
    description="REST API developed for the Web Applications course",
    version="1.0.0"
)
```

```
@app.get("/")
def root():
    return {
        "message": "FastAPI Workshop API",
        "version": "1.0.0"
    }
```

Start the application:

```
uvicorn app.main:app --reload
```

Open:

<http://127.0.0.1:8000/>

You should receive JSON:

```
{
  "message": "FastAPI Workshop API",
  "version": "1.0.0"
}
```

Part IV — OpenAPI documentation

6. Explore the automatic documentation

FastAPI automatically generates an **OpenAPI specification** from your routes and Pydantic models.

Open Swagger UI:

<http://127.0.0.1:8000/docs>

Also examine:

<http://127.0.0.1:8000/openapi.json>

The second URL contains the OpenAPI description of your API in JSON.

Your group must make the generated documentation understandable. Do not leave endpoints with unclear names or descriptions.

For example:

```
@app.get(
    "/public/info",
    tags=["Public"],
    summary="Obtain public API information",
    description="Returns general information without requiring authentication."
)
def public_info():
    return {
        "application": "Workshop API",
        "public": True
    }
```

Observe how this information appears in /docs.

Part V — Database

7. Configure SQLAlchemy

For example, using MySQL:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
import os

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "mysql+pymysql://apiuser:password@localhost/workshop"
)
```

```
engine = create_engine(DATABASE_URL)
```

```
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)
```

```
Base = declarative_base()
```

Do **not** commit real database passwords, JWT secrets, or social-login client secrets to Git.

Place them in .env.

Example:

```
DATABASE_URL=mysql+pymysql://apiuser:password@localhost/workshop
JWT_SECRET=replace-this-with-a-long-random-secret
ACCESS_TOKEN_MINUTES=15
REFRESH_TOKEN_DAYS=7
REMEMBER_REFRESH_TOKEN_DAYS=30
Add .env to .gitignore.
```

Part VI — User model

8. Create the Users table

At minimum, your users table must contain:

users

```
-----
id
username
full_name
email
password_hash
is_active
created_at
auth_provider
provider_user_id
```

A possible SQLAlchemy model is:

```
from sqlalchemy import (
    Column, Integer, String, Boolean, DateTime
)
from datetime import datetime
from app.database import Base
```

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, index=True)
    full_name = Column(String(150))
    email = Column(String(150), unique=True, index=True)
    password_hash = Column(String(255), nullable=True)

    is_active = Column(Boolean, default=True)

    auth_provider = Column(String(30), default="local")
    provider_user_id = Column(String(255), nullable=True)

    created_at = Column(DateTime, default=datetime.utcnow)
```

You may add fields relevant to your project, such as:

```
role
phone
department
profile_picture
last_login
```

For social-only accounts, password_hash may be NULL.

Part VII — Registration

9. Create a registration schema

Use Pydantic to validate incoming data.

```
from pydantic import BaseModel, EmailStr
```

```
class UserRegister(BaseModel):
```

```
    username: str
```

```
    full_name: str
```

```
    email: EmailStr
```

```
    password: str
```

```
Create:
```

```
POST /auth/register
```

Example request:

```
{
  "username": "student01",
  "full_name": "Ana Torres",
  "email": "ana@example.com",
  "password": "MySecurePassword123!"
}
```

Your endpoint must check that the username and email do not already exist.

Never store the plain-text password.

Store only a secure password hash.

Part VIII — JWT authentication

10. Understand the authentication process

Your authentication flow will be:

```
POST /auth/login
```

```
email/password
```

```
|
```

```
▼
```

```
Validate credentials
```

```
|
```

```
▼
```

```
Access Token + Refresh Token
```

```
|
```

```
▼
```

```
Client sends:
```

```
Authorization: Bearer <access-token>
```

```
|
```

```
▼
```

```
Protected endpoint
```

The access token should be relatively short-lived.

The refresh token should live longer and be used to obtain a new access token.

11. JWT token creation

Create security/jwt.py.

A simplified implementation can use python-jose:

```
from datetime import datetime, timedelta, timezone
from jose import jwt
```

```
SECRET_KEY = "CHANGE_THIS"
ALGORITHM = "HS256"
```

```
def create_access_token(user_id: int):
    expire = datetime.now(timezone.utc) + timedelta(minutes=15)

    payload = {
        "sub": str(user_id),
        "type": "access",
        "exp": expire
    }

    return jwt.encode(
        payload,
        SECRET_KEY,
        algorithm=ALGORITHM
    )
```

Do not leave the actual secret directly in your source code. Read it from .env.

Part IX — Login

12. Implement /auth/login

Create:

POST /auth/login

The login endpoint should accept:

```
{
    "email": "ana@example.com",
    "password": "MySecurePassword123!",
    "remember_me": false
}
```

After verifying the password, return something similar to:

```
{
  "access_token": "eyJ...",
  "refresh_token": "eyJ...",
  "token_type": "bearer",
  "expires_in": 900
}
```

An invalid password should produce an appropriate HTTP response such as:

401 Unauthorized

Do not return:

```
{
  "error": "Password is incorrect"
}
```

while using HTTP status 200.

Correct HTTP status codes are part of API design.

Part X — Remember Me

13. Implement Remember Me

remember_me must **not** mean that an access token never expires.

Instead, use different refresh-token lifetimes.

For example:

remember_me = false

Access token: 15 minutes

Refresh token: 7 days

remember_me = true

Access token: 15 minutes

Refresh token: 30 days

This maintains a short lifetime for access tokens while allowing a user to remain signed in for a longer period.

Part XI — Refresh tokens

14. Implement token refresh

Create:

POST /auth/refresh

Example:

```
{
  "refresh_token": "eyJ..."
}
```

The API must verify that:

- signature is valid
- token has not expired
- token type == refresh
- user still exists
- user is active
- token has not been revoked

Then issue a new access token.

For a stronger implementation, use **refresh-token rotation**: when a refresh token is successfully used, revoke it and issue a new refresh token.

Part XII — Logout

15. Implement logout correctly

JWT access tokens are normally stateless. Therefore, simply deleting a JWT on the server does not automatically invalidate a token already issued to a client.

For this workshop, maintain refresh-token/session information in the database.

For example:

refresh_tokens

- id
- user_id
- token_hash
- expires_at
- revoked_at
- created_at

Create:

POST /auth/logout

Logout should revoke the current refresh token/session.

For an optional advanced implementation, add a token identifier (jti) and maintain a denylist for access tokens when immediate invalidation is required.

Part XIII — Current authenticated user

16. Implement /users/me

Create:

GET /users/me

This endpoint requires:

Authorization: Bearer <access-token>

It should return:

```
{
  "id": 5,
  "username": "student01",
  "full_name": "Ana Torres",
  "email": "ana@example.com"
}
```

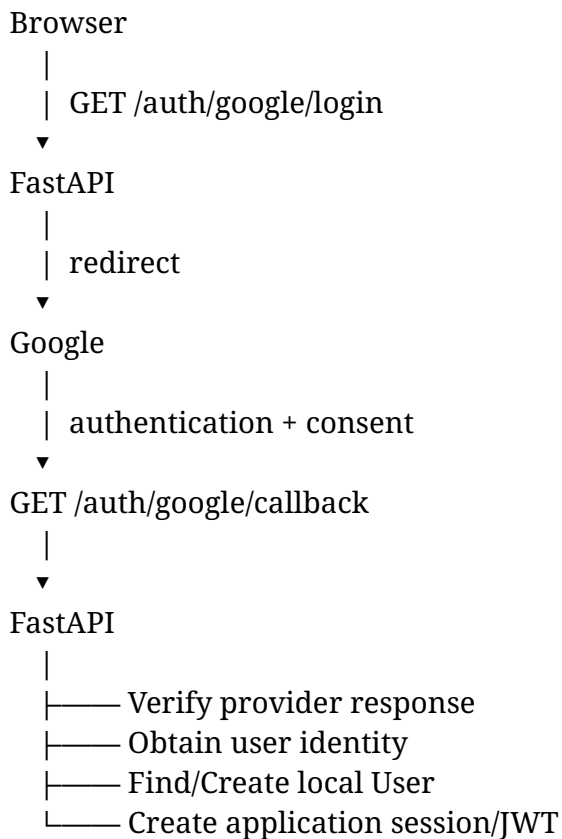
It must **never** return the password hash.

Part XIV — Social registration and login

17. Add one social authentication provider

Each group must integrate at least one provider that supports OAuth 2.0/OpenID Connect, for example Google Identity or GitHub OAuth.

A typical flow is:



For example, a Google user may produce a local record such as:

```
username    = ana@gmail.com
email       = ana@gmail.com
full_name   = Ana Torres
auth_provider = google
provider_user_id = provider-specific identifier
password_hash = NULL
```

Do not use the user's social-media password. Your API must never receive or store that password.

Part XV — Public endpoints

18. Create endpoints that do not require JWT

Your API must contain useful public endpoints.

Examples include:

```
GET /public/info
GET /public/categories
GET /public/products
GET /public/events
For example:
```

```
@app.get("/public/info", tags=["Public"])
def public_info():
    return {
        "api": "Workshop API",
        "version": "1.0",
        "authentication_required": False
    }
```

Verify that you can call the endpoint without logging in.

Part XVI — Your group's entity

19. Design your application entity

Now design at least one entity appropriate to your group's project.

For example:

Task

```
id
title
description
status
created_at
user_id
user_id establishes ownership:
```

User 1 ————— N Task

Implement:

```
POST /tasks
GET /tasks
GET /tasks/{id}
PUT /tasks/{id}
DELETE /tasks/{id}
```

You may instead use PATCH when only part of a resource is modified.

Part XVII — JWT authorization

20. Protect your CRUD endpoints

The previous CRUD operations must require authentication.

For example:

```
@app.get("/tasks")
def get_tasks(
    current_user = Depends(get_current_user)
):
    ...
```

Calling the endpoint without a JWT should fail.

Calling it with:

Authorization: Bearer eyJ...

should allow access if the JWT is valid.

Authentication answers:

Who is this user?

Authorization answers:

Is this user allowed to perform this operation?

Your API should address both questions.

For example, a user should normally not be allowed to delete another user's task merely by changing:

DELETE /tasks/17

to another ID.

Part XVIII — JSON responses

21. Use consistent responses

Successful creation could return:

```
{
    "id": 23,
    "title": "Prepare FastAPI workshop",
    "status": "pending",
    "user_id": 5
}
```

Use appropriate HTTP status codes:

200 OK

201 Created

204 No Content

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

422 Unprocessable Content

Do not return HTML pages from your REST API endpoints.

Part XIX — Improve OpenAPI documentation

22. Organize Swagger UI

Use tags:

```
tags=["Authentication"]
```

```
tags=["Users"]
```

```
tags=["Public"]
```

```
tags=["Tasks"]
```

Add response models:

```
@app.get(
    "/tasks/{task_id}",
    response_model=TaskResponse,
    tags=["Tasks"],
    summary="Get a task"
)
```

Use Pydantic schemas so OpenAPI can clearly show:

Request parameters

Request body

Required properties

Data types

Response structure

HTTP status codes

Authentication requirements

Your /docs page should look like usable API documentation, not simply a collection of undocumented routes.

Part XX — Test authentication with Swagger UI

23. Test through /docs

Open:

```
http://127.0.0.1:8000/docs
```

First test your public endpoints.

Then test:

```
POST /auth/register
```

followed by:

POST /auth/login

Copy the resulting access token.

Configure FastAPI's OpenAPI security scheme so that Swagger UI displays the **Authorize** button.

After authorization, call:

GET /users/me

Then test your protected CRUD endpoints.

Also deliberately test an invalid or expired JWT and record the response.

Part XXI — Test with Postman

24. Create a Postman collection

Create a collection named:

FastAPI Workshop

Define a collection variable:

base_url = http://127.0.0.1:8000

Your requests can then use:

{{base_url}}/auth/register

{{base_url}}/auth/login

{{base_url}}/users/me

{{base_url}}/tasks

25. Store the JWT automatically

After executing the login request, configure Postman to store the returned access token in a variable.

For subsequent requests use Bearer Token authentication with:

{{access_token}}

Your HTTP request will then contain:

Authorization: Bearer {{access_token}}

Do the same with:

{{refresh_token}}

when testing /auth/refresh.

Part XXII — Required tests

26. Perform the following tests

Your group must demonstrate all of these cases:

Test	Expected result
Public endpoint without JWT	Success
Registration with valid information	User created
Duplicate email	Rejected
Invalid email	Rejected
Login with correct password	Tokens returned
Login with incorrect password	401
/users/me without token	Rejected
/users/me with valid token	User returned
Protected CRUD without JWT	Rejected
Protected CRUD with JWT	Success
Access another user's protected resource	Rejected
Expired/invalid token	Rejected
Refresh valid token	New token issued
Invalid refresh token	Rejected
Logout	Session/refresh token revoked
Use revoked refresh token	Rejected
Remember Me = false	Normal refresh lifetime
Remember Me = true	Extended refresh lifetime
Social registration/login	User authenticated
OpenAPI/Swagger test	Success
Postman test	Success

Do not test only successful requests. A secure API must also be tested against invalid requests.

Part XXIII — Minimum endpoint specification

Your finished project should contain at least the following conceptual endpoints:

PUBLIC

GET /public/info

GET /public/<resource>

AUTHENTICATION

POST /auth/register

POST /auth/login
POST /auth/refresh
POST /auth/logout

SOCIAL AUTHENTICATION

GET /auth/<provider>/login
GET /auth/<provider>/callback

USERS

GET /users/me

PROTECTED ENTITY

POST /<entities>
GET /<entities>
GET /<entities>/{id}
PUT /<entities>/{id}
DELETE /<entities>/{id}

You may add additional endpoints appropriate to your application.

Part XXIV — Security requirements

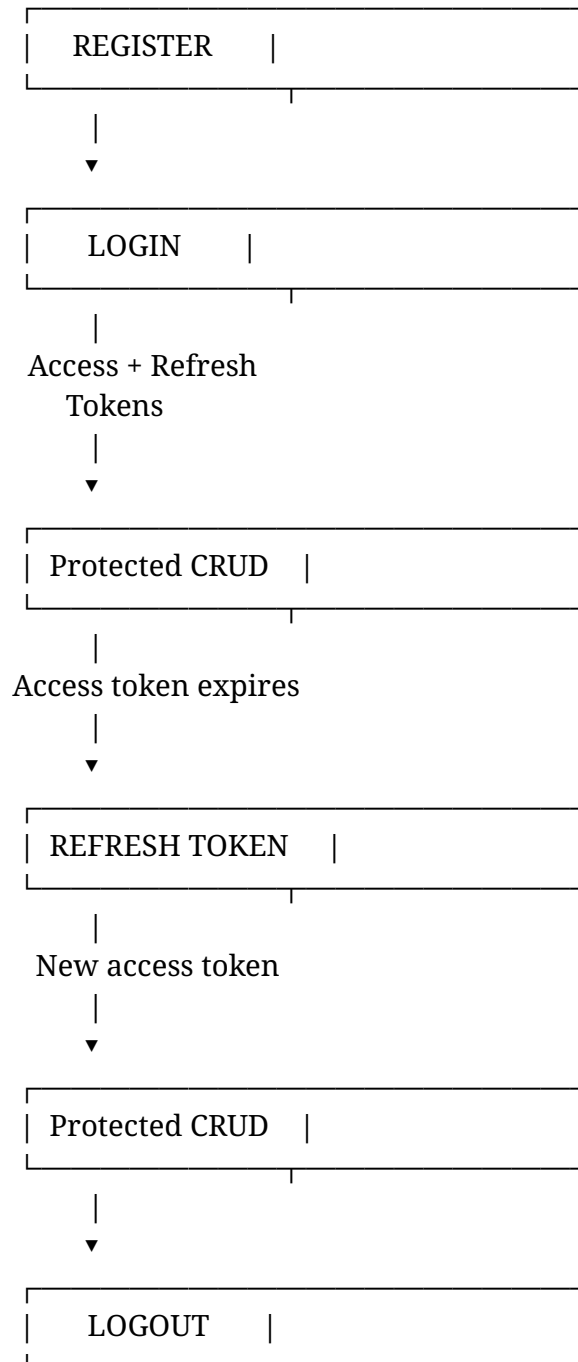
Passwords, JWTs, refresh tokens, OAuth credentials, and database credentials are security-sensitive data. Your project must therefore follow these rules:

- Never store plain-text passwords.
- Never return password hashes through the API.
- Keep secrets outside the source code.
- Do not commit .env to Git.
- Validate JWT signatures and expiration.
- Distinguish access tokens from refresh tokens.
- Revoke refresh tokens during logout.
- Validate ownership/permissions before modifying resources.
- Validate request data using Pydantic.
- Do not trust a user_id supplied by the client when ownership can be obtained from the authenticated JWT.
- In a real deployment, use HTTPS.

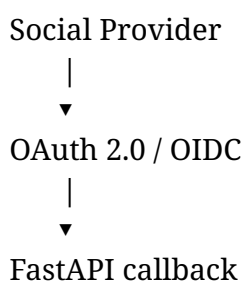
For production-quality password storage, use a modern password-hashing algorithm such as **Argon2id**; bcrypt is also common in existing systems.

Part XXV — Final API workflow

Your completed system should support this sequence:



The alternative authentication path is:





Create/find User



Application JWT



Protected API